

SOFTWARE FOUNDATIONS

VOLUME 1: LOGICAL FOUNDATIONS

TABLE OF CONTENTS

INDEX

ROADMAP

TACTICS

MORE BASIC TACTICS

This chapter introduces several additional proof strategies and tactics that allow us to begin proving more interesting properties of functional programs.

We will see:

- how to use auxiliary lemmas in both "forward-" and "backward-style" proofs;
- how to reason about data constructors -- in particular, how to use the fact that they are injective and disjoint;
- how to strengthen an induction hypothesis, and when such strengthening is required; and
- more details on how to reason by case analysis.

From [LF Require Export Poly](#).

The `apply` Tactic

We often encounter situations where the goal to be proved is *exactly* the same as some hypothesis in the context or some previously proved lemma.

```
Theorem silly1 : ∀ (n m : nat),  
  n = m →  
  n = m.  
Proof.  
  intros n m eq.
```

Here, we could finish with "`rewrite → eq. reflexivity.`" as we have done several times before. Or we can finish in a single step by using `apply`:

```
  apply eq. Qed.
```

The `apply` tactic also works with *conditional* hypotheses and lemmas: if the statement being applied is an implication, then the premises of this implication will be added to the list of subgoals needing to be proved. `apply` also works with *conditional* hypotheses:

```

Theorem silly2 :  $\forall$  (n m o p : nat),
  n = m  $\rightarrow$ 
  (n = m  $\rightarrow$  [n;o] = [m;p])  $\rightarrow$ 
  [n;o] = [m;p].
Proof.
  intros n m o p eq1 eq2.
  apply eq2. apply eq1. Qed.

```

Typically, when we use `apply H`, the statement `H` will begin with a $\forall$ that introduces some *universally quantified variables*.

When Rocq matches the current goal against the conclusion of `H`, it will try to find appropriate values for these variables. For example, when we do `apply eq2` in the following proof, the universal variable `q` in `eq2` gets instantiated with `n`, and `r` gets instantiated with `m`.

```

Theorem silly2a :  $\forall$  (n m : nat),
  (n,n) = (m,m)  $\rightarrow$ 
  ( $\forall$  (q r : nat), (q,q) = (r,r)  $\rightarrow$  [q] = [r])  $\rightarrow$ 
  [n] = [m].
Proof.
  intros n m eq1 eq2.
  apply eq2. apply eq1. Qed.

```

Exercise: 2 stars, standard, optional (silly_ex)

Complete the following proof using only `intros` and `apply`.

```

Theorem silly_ex :  $\forall$  p,
  ( $\forall$  n, even n = true  $\rightarrow$  even (S n) = false)  $\rightarrow$ 
  ( $\forall$  n, even n = false  $\rightarrow$  odd n = true)  $\rightarrow$ 
  even p = true  $\rightarrow$ 
  odd (S p) = true.
Proof.
  (* FILL IN HERE *) Admitted.

```

□

To use the `apply` tactic, the (conclusion of the) fact being applied must match the goal exactly (perhaps after simplification) -- for example, `apply` will not work if the left and right sides of the equality are swapped.

```

Theorem silly3 :  $\forall$  (n m : nat),
  n = m  $\rightarrow$ 
  m = n.
Proof.
  intros n m H.

```

Here we cannot use `apply` directly...

```
Fail apply H.
```

...but we can use the `symmetry` tactic, which switches the left and right sides of an equality in the goal.

```
symmetry. apply H. Qed.
```

Exercise: 2 stars, standard (apply_exercise1)

You can use `apply` with previously defined theorems, not just hypotheses in the context. Use `Search` to find a previously-defined theorem about `rev` from `Lists`. Use that theorem as part of your (relatively short) solution to this exercise. You do not need `induction`.

```
Theorem rev_exercise1 : ∀ (l l' : list nat),
  l = rev l' →
  l' = rev l.
Proof.
  (* FILL IN HERE *) Admitted.
□
```

Exercise: 1 star, standard, optional (apply rewrite)

Briefly explain the difference between the tactics `apply` and `rewrite`. What are the situations where both can usefully be applied?

```
(* FILL IN HERE *)
□
```

The `apply` with Tactic

The following silly example uses two rewrites in a row to get from `[a;b]` to `[e;f]`.

```
Example trans_eq_example : ∀ (a b c d e f : nat),
  [a;b] = [c;d] →
  [c;d] = [e;f] →
  [a;b] = [e;f].
Proof.
  intros a b c d e f eq1 eq2.
  rewrite → eq1. apply eq2. Qed.
```

Since this is a common pattern, we might like to pull it out as a lemma that records, once and for all, the fact that equality is transitive.

```
Theorem trans_eq : ∀ (X:Type) (x y z : X),
  x = y → y = z → x = z.
Proof.
  intros X x y z eq1 eq2. rewrite → eq1. rewrite → eq2.
  reflexivity. Qed.
```

Now, we should be able to use `trans_eq` to prove the above example. However, to do this we need a slight refinement of the `apply` tactic.

```
Example trans_eq_example' : ∀ (a b c d e f : nat),
  [a;b] = [c;d] →
  [c;d] = [e;f] →
  [a;b] = [e;f].
Proof.
  intros a b c d e f eq1 eq2.
```

If we simply tell Rocq `apply trans_eq` at this point, it can tell (by matching the goal against the conclusion of the lemma) that it should instantiate `X` with `[nat]`, `x` with `[a,b]`, and `z` with `[e,f]`. However, the matching

process doesn't determine an instantiation for y : we have to supply one explicitly by adding "with ($y := [c; d]$)" to the invocation of `apply`.

```
apply trans_eq with (y:=[c;d]).
apply eq1. apply eq2. Qed.
```

Actually, the name y in the `with` clause is not required, since Rocq is often smart enough to figure out which variable we are instantiating. We could instead simply write `apply trans_eq with [c;d]`.

Rocq also has a built-in tactic `transitivity` that accomplishes the same purpose as applying `trans_eq`. The tactic requires us to state the instantiation we want, just like `apply with` does.

```
Example trans_eq_example'' : ∀ (a b c d e f : nat),
  [a;b] = [c;d] →
  [c;d] = [e;f] →
  [a;b] = [e;f].
Proof.
  intros a b c d e f eq1 eq2.
  transitivity [c;d].
  apply eq1. apply eq2. Qed.
```

Exercise: 3 stars, standard, optional (trans eq exercise)

```
Example trans_eq_exercise : ∀ (n m o p : nat),
  m = (minustwo o) →
  (n + p) = m →
  (n + p) = (minustwo o).
Proof.
  (* FILL IN HERE *) Admitted.
□
```

The injection and discriminate Tactics

Recall the definition of natural numbers:

```
Inductive nat : Type :=
  | 0
  | S (n : nat).
```

It is obvious from this definition that every number has one of two forms: either it is the constructor `0` or it is built by applying the constructor `S` to another number. But there is more here than meets the eye: implicit in the definition are two additional facts:

- The constructor `S` is *injective* (or *one-to-one*). That is, if $S\ n = S\ m$, it must also be that $n = m$.
- The constructors `0` and `S` are *disjoint*. That is, `0` is not equal to `S n` for any n .

Similar principles apply to every inductively defined type: all constructors are injective, and the values built from distinct constructors are never equal. For

lists, the `cons` constructor is injective and the empty list `nil` is different from every non-empty list. For booleans, `true` and `false` are different. (Since `true` and `false` take no arguments, their injectivity is neither here nor there.) And so on.

We can *prove* the injectivity of `S` by using the `pred` function defined in `Basics.v`.

```
Theorem S_injective : ∀ (n m : nat),
  S n = S m →
  n = m.
Proof.
  intros n m H1.
  assert (H2: n = pred (S n)). { reflexivity. }
  rewrite H2. rewrite H1. simpl. reflexivity.
Qed.
```

Rocq's `assert` tactic, used above, adds the given hypothesis to the context, but it first requires you to prove the hypothesis as a new goal.

This technique for injectivity can be generalized to any constructor by writing the equivalent of `pred` -- i.e., writing a function that "undoes" one application of the constructor.

As a convenient alternative, Rocq provides a tactic called `injection` that allows us to exploit the injectivity of any constructor. Here is an alternate proof of the above theorem using `injection`:

```
Theorem S_injective' : ∀ (n m : nat),
  S n = S m →
  n = m.
Proof.
  intros n m H.
```

By writing `injection H as Hmn` at this point, we are asking Rocq to generate all equations that it can infer from `H` using the injectivity of constructors (in the present example, the equation `n = m`). Each such equation is added as a hypothesis (called `Hmn` in this case) into the context.

```
  injection H as Hnm. apply Hnm.
Qed.
```

Here's a more interesting example that shows how `injection` can derive multiple equations at once.

```
Theorem injection_ex1 : ∀ (n m o : nat),
  [n;m] = [o;o] →
  n = m.
Proof.
  intros n m o H.
  (* WORKED IN CLASS *)
  injection H as H1 H2.
  rewrite H1. rewrite H2. reflexivity.
Qed.
```

Exercise: 3 stars, standard (injection_ex₃)

```

Example injection_ex3 : ∀ (X : Type) (x y z : X) (l j : list X),
  x :: y :: l = z :: j →
  j = z :: l →
  x = y.
Proof.
  (* FILL IN HERE *) Admitted.
□

```

So much for injectivity of constructors. What about disjointness?

The principle of disjointness says that two terms beginning with different constructors (like 0 and S, or true and false) can never be equal. This means that, any time we find ourselves in a context where we've *assumed* that two such terms are equal, we are justified in concluding anything we want, since the assumption is nonsensical.

The `discriminate` tactic embodies this principle: It is used on a hypothesis involving an equality between different constructors (e.g., `false = true`), and it solves the current goal immediately. Some examples:

```

Theorem discriminate_ex1 : ∀ (n m : nat),
  false = true →
  n = m.
Proof.
  intros n m contra. discriminate contra. Qed.

Theorem discriminate_ex2 : ∀ (n : nat),
  S n = 0 →
  2 + 2 = 5.
Proof.
  intros n contra. discriminate contra. Qed.

```

These examples are instances of a logical principle known as the *principle of explosion*, which asserts that a contradictory hypothesis entails anything (even manifestly false things!).

If you find the principle of explosion confusing, remember that these proofs are *not* showing that the conclusion of the statement holds. Rather, they are showing that, *if* the nonsensical situation described by the premise did somehow hold, *then* the nonsensical conclusion would too -- because we'd be living in an inconsistent universe where every statement is true.

We'll explore the principle of explosion in more detail in the next chapter.

Exercise: 1 star, standard (discriminate_ex3)

```

Example discriminate_ex3 :
  ∀ (X : Type) (x y z : X) (l j : list X),
    x :: y :: l = [] →
    x = z.
Proof.
  (* FILL IN HERE *) Admitted.
□

```

For a more useful example, we can use `discriminate` to make a connection between the two different notions of equality (`=` and `=?`) that we

have seen for natural numbers.

```
Theorem eqb_0_l : ∀ n,  
  0 =? n = true → n = 0.  
Proof.  
  intros n.
```

We can proceed by case analysis on n . The first case is trivial.

```
destruct n as [| n'] eqn:E.  
- (* n = 0 *)  
  intros H. reflexivity.
```

However, the second one doesn't look so simple: assuming $0 =? (S\ n') = \text{true}$, we must show $S\ n' = 0$! The way forward is to observe that the assumption itself is nonsensical:

```
- (* n = S n' *)  
  simpl.
```

If we use `discriminate` on this hypothesis, Rocq confirms that the subgoal we are working on is impossible and removes it from further consideration.

```
  intros H. discriminate H.  
Qed.
```

The injectivity of constructors allows us to reason that $\forall (n\ m : \text{nat}), S\ n = S\ m \rightarrow n = m$. The converse of this implication is an instance of a more general fact about both constructors and functions, which we will find useful below:

```
Theorem f_equal : ∀ (A B : Type) (f: A → B) (x y: A),  
  x = y → f x = f y.  
Proof. intros A B f x y eq. rewrite eq. reflexivity. Qed.  
  
Theorem eq_implies_succ_equal : ∀ (n m : nat),  
  n = m → S n = S m.  
Proof. intros n m H. apply f_equal. apply H. Qed.
```

Indeed, there is also a tactic named `f_equal` that can prove such theorems directly. Given a goal of the form $f\ a_1 \dots a_n = g\ b_1 \dots b_n$, the tactic `f_equal` will produce subgoals of the form $f = g, a_1 = b_1, \dots, a_n = b_n$. At the same time, any of these subgoals that are simple enough (e.g., immediately provable by `reflexivity`) will be automatically discharged.

```
Theorem eq_implies_succ_equal' : ∀ (n m : nat),  
  n = m → S n = S m.  
Proof. intros n m H. f_equal. apply H. Qed.
```

Using Tactics on Hypotheses

By default, most tactics work on the goal formula and leave the context unchanged. However, most tactics also have a variant that performs a similar operation on a statement in the context.

For example, the tactic `simpl in H` performs simplification on the hypothesis H in the context.

```

Theorem S_inj : ∀ (n m : nat) (b : bool),
  ((S n) =? (S m)) = b →
  (n =? m) = b.
Proof.
  intros n m b H. simpl in H. apply H. Qed.

```

Similarly, `apply L in H` matches some conditional statement `L` (of the form $X \rightarrow Y$, say) against a hypothesis `H` in the context. However, unlike ordinary `apply` (which rewrites a goal matching `Y` into a subgoal `X`), `apply L in H` matches `H` against `X` and, if successful, replaces it with `Y`.

In other words, `apply L in H` gives us a form of "forward reasoning": given $X \rightarrow Y$ and a hypothesis matching `X`, it produces a hypothesis matching `Y`.

By contrast, `apply L` is "backward reasoning": it says that if we know $X \rightarrow Y$ and we are trying to prove `Y`, it suffices to prove `X`.

Here is a variant of a proof from above, using forward reasoning throughout instead of backward reasoning.

```

Theorem silly4 : ∀ (n m p q : nat),
  (n = m → p = q) →
  m = n →
  q = p.
Proof.
  intros n m p q EQ H.
  symmetry in H. apply EQ in H. symmetry in H.
  apply H. Qed.

```

Forward reasoning starts from what is *given* (premises, previously proven theorems) and iteratively draws conclusions from them until the goal is reached. Backward reasoning starts from the *goal* and iteratively reasons about what would imply the goal, until premises or previously proven theorems are reached.

The informal proofs seen in math or computer science classes tend to use forward reasoning. By contrast, idiomatic use of Rocq generally favors backward reasoning, though in some situations the forward style can be easier to think about.

Specializing Hypotheses

Another handy tactic for manipulating hypotheses is `specialize`. It is essentially just a combination of `assert` and `apply`, but it often provides a pleasingly smooth way to nail down overly general assumptions. It works like this:

If `H` is a quantified hypothesis in the current context -- i.e., $H : \forall (x:T), P$ -- then `specialize H with (x := e)` will change `H` so that it looks like `P` with `x` replaced by `e`.

For example:

```

Theorem specialize_example: ∀ n,
  (∀ m, m×n = 0)

```



```

→ n = 0.
Proof.
  intros n H.
  specialize H with (m := 1).
  rewrite mult_1_l in H.
  apply H. Qed.

```

Exercise: 3 stars, standard (nth_error always none)

Use `specialize` to prove the the following lemma, following the model of `specialize_example` above. Do not use `induction`.

```

Lemma nth_error_always_none: ∀ (l : list nat),
  (∀ i, nth_error l i = None) →
  l = [].
Proof.
  (* FILL IN HERE *) Admitted.

```

□

Using `specialize` before `apply` gives us yet another way to control where `apply` does its work.

```

Example trans_eq_example''' : ∀ (a b c d e f : nat),
  [a;b] = [c;d] →
  [c;d] = [e;f] →
  [a;b] = [e;f].
Proof.
  intros a b c d e f eq1 eq2.
  specialize trans_eq with (y:=[c;d]) as H.
  apply H.
  apply eq1.
  apply eq2. Qed.

```

Things to note:

- We can `specialize` facts in the global context, not just local hypotheses.
- The `as ...` clause at the end tells `specialize` how to name the new hypothesis in this case.

Varying the Induction Hypothesis

Sometimes it is important to control the exact form of the induction hypothesis when carrying out inductive proofs in Rocq. In particular, we may need to be careful about which of the assumptions we move (using `intros`) from the goal to the context before invoking the `induction` tactic.

For example, suppose we want to show that `double` is injective -- i.e., that it maps different arguments to different results:

```

Theorem double_injective: ∀ n m,
  double n = double m →
  n = m.

```

The way we start this proof is a bit delicate: if we begin it with

```

intros n. induction n.

```

then all will be well. But if we begin it with introducing *both* variables

```
intros n m. induction n.
```

we get stuck in the middle of the inductive case...

```
Theorem double_injective_FAILED : ∀ n m,  
  double n = double m →  
  n = m.
```

```
Proof.
```

```
  intros n m. induction n as [| n' IHn'].  
  - (* n = 0 *) simpl. intros eq. destruct m as [| m'] eqn:E.  
    + (* m = 0 *) reflexivity.  
    + (* m = S m' *) discriminate eq.  
  - (* n = S n' *) intros eq. destruct m as [| m'] eqn:E.  
    + (* m = 0 *) discriminate eq.  
    + (* m = S m' *) f_equal.
```

At this point, the induction hypothesis (IHn') does *not* give us $n' = m'$ -- there is an extra *S* in the way -- so the goal is not provable.

```
Abort.
```

What went wrong?

The problem is that, at the point where we invoke the induction hypothesis, we have already introduced *m* into the context -- intuitively, we have told Rocq, "Let's consider some particular *n* and *m*..." and we now have to prove that, if $\text{double } n = \text{double } m$ for *these particular* *n* and *m*, then $n = m$.

The next tactic, `induction n` says to Rocq: We are going to show the goal by induction on *n*. That is, we are going to prove, for *all* *n*, that the proposition

- $P\ n = \text{"if } \text{double } n = \text{double } m, \text{ then } n = m\text{"}$

holds, by showing

- $P\ 0$
(i.e., "if $\text{double } 0 = \text{double } m$ then $0 = m$ ") and
- $P\ n \rightarrow P\ (S\ n)$
(i.e., "if $\text{double } n = \text{double } m$ then $n = m$ " implies "if $\text{double } (S\ n) = \text{double } m$ then $S\ n = m$ ").

If we look closely at the second statement, it is saying something rather strange: that, for a *particular* *m*, if we know

- "if $\text{double } n = \text{double } m$ then $n = m$ "

then we can prove

- "if $\text{double } (S\ n) = \text{double } m$ then $S\ n = m$ ".

To see why this is strange, let's think of a particular *m* -- say, **5**. The statement is then saying that, if we know

- $Q = \text{"if } \text{double } n = 10 \text{ then } n = 5\text{"}$

then we can prove

- $R = \text{"if } \text{double } (S\ n) = 10 \text{ then } S\ n = 5\text{"}$.

But knowing Q doesn't give us any help at all with proving R ! If we tried to prove R from Q , we would start with something like "Suppose $\text{double } (S\ n) = 10\ldots$ " but then we'd be stuck: knowing that $\text{double } (S\ n)$ is 10 tells us nothing helpful about whether $\text{double } n$ is 10 (indeed, it strongly suggests that $\text{double } n$ is *not* 10!!), so Q is useless.

Trying to carry out this proof by induction on n when m is already in the context doesn't work because we are then trying to prove a statement involving *every* n but just a *particular* m .

A successful proof of `double_injective` keeps m universally quantified in the goal statement at the point where the `induction` tactic is invoked on n :

```
Theorem double_injective : ∀ n m,
  double n = double m →
  n = m.
Proof.
  intros n. induction n as [| n' IHn'].
  - (* n = 0 *) simpl. intros m eq. destruct m as [| m'] eqn:E.
    + (* m = 0 *) reflexivity.
    + (* m = S m' *) discriminate eq.
  - (* n = S n' *)
```

Notice that both the goal and the induction hypothesis are different this time: the goal asks us to prove something more general (i.e., we must prove the statement for *every* m), but the induction hypothesis IH' is correspondingly more flexible, allowing us to choose any m we like when we apply it.

```
intros m eq.
```

Now we've chosen a particular m and introduced the assumption that $\text{double } n = \text{double } m$. Since we are doing a case analysis on n , we also need a case analysis on m to keep the two in sync.

```
destruct m as [| m'] eqn:E.
+ (* m = 0 *)
```

The 0 case is trivial:

```
discriminate eq.
+ (* m = S m' *)
  f_equal.
```

Since we are now in the second branch of the `destruct m`, the m' mentioned in the context is the predecessor of the m we started out talking about. Since we are also in the S branch of the induction, this is perfect: if we instantiate the generic m in the IH with the current m' (this instantiation is performed automatically by the `apply` in the next step), then IHn' gives us exactly what we need to finish the proof.

```
apply IHn'. simpl in eq. injection eq as goal. apply
goal. Qed.
```

The thing to take away from all this is that you need to be careful, when using induction, that you are not trying to prove something too specific: When proving a property quantified over variables n and m by induction on n , it is sometimes crucial to leave m "generic."

The following exercise, which further strengthens the link between $=?$ and $=$, follows the same pattern.

Exercise: 2 stars, standard (eqb_true)

```
Theorem eqb_true : ∀ n m,
  n =? m = true → n = m.
Proof.
  (* FILL IN HERE *) Admitted.
□
```

Exercise: 2 stars, advanced, optional (eqb_true_informal)

Give a careful informal proof of `eqb_true`, stating the induction hypothesis explicitly and being as explicit as possible about quantifiers, everywhere.

```
(* FILL IN HERE *)

(* Do not modify the following line: *)
Definition manual_grade_for_informal_proof : option
(nat×string) := None.
□
```

Exercise: 3 stars, standard, especially useful (plus_n_n_injective)

In addition to being careful about how you use `intros`, practice using "in" variants in this proof. (Hint: use `plus_n_Sm`.)

```
Theorem plus_n_n_injective : ∀ n m,
  n + n = m + m →
  n = m.
Proof.
  (* FILL IN HERE *) Admitted.
□
```

The strategy of doing fewer `intros` before an `induction` to obtain a more general IH doesn't always work; sometimes some *rearrangement* of quantified variables is needed. Suppose, for example, that we wanted to prove `double_injective` by induction on m instead of n .

```
Theorem double_injective_take2_FAILED : ∀ n m,
  double n = double m →
  n = m.
Proof.
  intros n m. induction m as [| m' IHm'].
  - (* m = 0 *) simpl. intros eq. destruct n as [| n'] eqn:E.
    + (* n = 0 *) reflexivity.
    + (* n = S n' *) discriminate eq.
  - (* m = S m' *) intros eq. destruct n as [| n'] eqn:E.
    + (* n = 0 *) discriminate eq.
    + (* n = S n' *) f_equal.
    (* We are stuck here, just like before. *)
Abort.
```

The problem is that, to do induction on m , we must first introduce n . (If we simply say `induction m` without introducing anything first, Rocq will automatically introduce n for us!)

What can we do about this? One possibility is to rewrite the statement of the lemma so that m is quantified before n . This works, but it's not nice: We don't want to have to twist the statements of lemmas to fit the needs of a particular strategy for proving them! Rather we want to state them in the clearest and most natural way.

What we can do instead is to first introduce all the quantified variables and then *re-generalize* one or more of them, selectively taking variables out of the context and putting them back at the beginning of the goal. The `generalize dependent` tactic does this.

```
Theorem double_injective_take2 : ∀ n m,
  double n = double m →
  n = m.
Proof.
  intros n m.
  (* n and m are both in the context *)
  generalize dependent n.
  (* Now n is back in the goal and we can do induction on
     m and get a sufficiently general IH. *)
  induction m as [| m' IHm'].
  - (* m = 0 *) simpl. intros n eq. destruct n as [| n'] eqn:E.
    + (* n = 0 *) reflexivity.
    + (* n = S n' *) discriminate eq.
  - (* m = S m' *) intros n eq. destruct n as [| n'] eqn:E.
    + (* n = 0 *) discriminate eq.
    + (* n = S n' *) f_equal.
    apply IHm'. injection eq as goal. apply goal. Qed.
```

Let's look at an informal proof of this theorem. Note that the proposition we prove by induction leaves n quantified, corresponding to the use of `generalize dependent` in our formal proof.

Theorem: For any nats n and m , if `double n = double m`, then $n = m$.

Proof: Let m be a nat. We prove by induction on m that, for any n , if `double n = double m` then $n = m$.

- First, suppose $m = 0$, and suppose n is a number such that `double n = double m`. We must show that $n = 0$.

Since $m = 0$, by the definition of `double` we have `double n = 0`. There are two cases to consider for n . If $n = 0$ we are done, since $m = 0 = n$, as required. Otherwise, if $n = S n'$ for some n' , we derive a contradiction: by the definition of `double`, we can calculate `double n = S (S (double n'))`, but this contradicts the assumption that `double n = 0`.

- Second, suppose $m = S m'$ and that n is again a number such that `double n = double m`. We must show that $n = S m'$, with the

induction hypothesis that for every number s , if $\text{double } s = \text{double } m'$ then $s = m'$.

By the fact that $m = S \ m'$ and the definition of `double`, we have $\text{double } n = S \ (S \ (\text{double } m'))$. There are two cases to consider for n .

If $n = 0$, then by definition $\text{double } n = 0$, a contradiction.

Thus, we may assume that $n = S \ n'$ for some n' , and again by the definition of `double` we have $S \ (S \ (\text{double } n')) = S \ (S \ (\text{double } m'))$, which implies by injectivity that $\text{double } n' = \text{double } m'$.

Instantiating the induction hypothesis with n' thus allows us to conclude that $n' = m'$, and it follows immediately that $S \ n' = S \ m'$.

Since $S \ n' = n$ and $S \ m' = m$, this is just what we wanted to show. $\square$

Rewriting with conditional statements

Suppose that we want to show that `plus` is the inverse of `minus`. Since we are working with natural numbers, we need an assumption to prevent `minus` from truncating its result. With this assumption, the induction hypothesis becomes $\forall m, n' \leq m \Rightarrow m - n' + n' = m$. The beginning of the proof uses techniques we have already seen -- in particular, notice how we induct on n before introducing m , so that the induction hypothesis becomes sufficiently general.

`Lemma sub_add_leb :  $\forall n \ m, n \leq m \Rightarrow m - n + n = m$ .`

`Proof.`

```

intros n.
induction n as [| n' IHn'].
- (* n = 0 *)
  intros m H. rewrite add_0_r. destruct m as [| m'].
  + (* m = 0 *)
    reflexivity.
  + (* m = S m' *)
    reflexivity.
- (* n = S n' *)
  intros m H. destruct m as [| m'].
  + (* m = 0 *)
    discriminate.
  + (* m = S m' *)
    simpl in H. simpl. rewrite <- plus_n_Sm.
```

At this point, we need to show $S \ ((m' - n') + n') = S \ m'$ from the assumption $(n' \leq m') = \text{true}$. We could use the `assert` tactic to prove $(m' - n') + n' = m'$ from the induction hypothesis. However, we can also just use `rewrite` directly: if we rewrite with a conditional statement of the form $P \rightarrow a = b$, then `Rocq` tries to rewrite with $a = b$, and then asks us to prove P in a new subgoal. If the statement has more than one assumption, then we get one subgoal for each assumption.

```

    rewrite IHn'.
    × reflexivity.
    × apply H.
Qed.

```

Exercise: 3 stars, standard, especially useful (gen_dep_practice)

Prove this by induction on `l`.

```

Theorem nth_error_after_last: ∀ (n : nat) (X : Type) (l : list
X),
  length l = n →
  nth_error l n = None.
Proof.
  (* FILL IN HERE *) Admitted.
□

```

Unfolding Definitions

It sometimes happens that we need to manually unfold a name that has been introduced by a `Definition` so that we can manipulate the expression it stands for.

For example, if we define...

```

Definition square n := n × n.

```

...and try to prove a simple fact about `square`...

```

Lemma square_mult : ∀ n m, square (n × m) = square n × square
m.
Proof.
  intros n m.
  simpl.

```

...we appear to be stuck: `simpl` doesn't simplify anything, and since we haven't proved any other facts about `square`, there is nothing we can `apply` or `rewrite` with.

To make progress, we can manually unfold the definition of `square`:

```

  unfold square.

```

Now we have plenty to work with: both sides of the equality are expressions involving multiplication, and we have lots of facts about multiplication at our disposal. In particular, we know that it is commutative and associative, and from these it is not hard to finish the proof.

```

  rewrite mult_assoc.
  assert (H : n × m × n = n × n × m).
  { rewrite mul_comm. apply mult_assoc. }
  rewrite H. rewrite mult_assoc. reflexivity.
Qed.

```

At this point, a bit deeper discussion of unfolding and simplification is in order.

We already have observed that tactics like `simpl`, `reflexivity`, and `apply` will often unfold the definitions of functions automatically when this

allows them to make progress. For example, if we define `foo m` to be the constant 5...

```
Definition foo (x: nat) := 5.
```

... then the `simpl` in the following proof (or the `reflexivity`, if we omit the `simpl`) will unfold `foo m` to `(fun x => 5) m` and further simplify this expression to just 5.

```
Fact silly_fact_1 : ∀ m, foo m + 1 = foo (m + 1) + 1.
```

```
Proof.
```

```
  intros m.
```

```
  simpl.
```

```
  reflexivity.
```

```
Qed.
```

But this automatic unfolding is somewhat conservative. For example, if we define a slightly more complicated function involving a pattern match...

```
Definition bar x :=  
  match x with  
  | 0 => 5  
  | S _ => 5  
end.
```

...then the analogous proof will get stuck:

```
Fact silly_fact_2_FAILED : ∀ m, bar m + 1 = bar (m + 1) + 1.
```

```
Proof.
```

```
  intros m.
```

```
  simpl. (* Does nothing! *)
```

```
Abort.
```

The reason that `simpl` doesn't make progress here is that it notices that, after tentatively unfolding `bar m`, it is left with a match whose scrutinee, `m`, is a variable, so the match cannot be simplified further. It is not smart enough to notice that the two branches of the match are identical, so it gives up on unfolding `bar m` and leaves it alone.

Similarly, tentatively unfolding `bar (m+1)` leaves a match whose scrutinee is a function application (that cannot itself be simplified, even after unfolding the definition of `+`), so `simpl` leaves it alone.

At this point, there are two ways to make progress. One is to use `destruct m` to break the proof into two cases, each focusing on a more concrete choice of `m` (`0` vs `S _`). In each case, the match inside of `bar` can now make progress, and the proof is easy to complete.

```
Fact silly_fact_2 : ∀ m, bar m + 1 = bar (m + 1) + 1.
```

```
Proof.
```

```
  intros m.
```

```
  destruct m eqn:E.
```

```
  - simpl. reflexivity.
```

```
  - simpl. reflexivity.
```

```
Qed.
```

This approach works, but it depends on our recognizing that the match hidden inside `bar` is what was preventing us from making progress.

A more straightforward way forward is to explicitly tell Rocq to unfold `bar`.


```
Fact silly_fact_2' : ∀ m, bar m + 1 = bar (m + 1) + 1.
Proof.
  intros m.
  unfold bar.
```

Now it is apparent that we are stuck on the `match` expressions on both sides of the `=`, and we can use `destruct` to finish the proof without thinking so hard.

```
destruct m eqn:E.
- reflexivity.
- reflexivity.
Qed.
```

Using `destruct` on Compound Expressions

We have seen many examples where `destruct` is used to perform case analysis of the value of some variable. Sometimes we need to reason by cases on the result of some *expression*. We can also do this with `destruct`.

Here are some examples:

```
Definition sillyfun (n : nat) : bool :=
  if n =? 3 then false
  else if n =? 5 then false
  else false.

Theorem sillyfun_false : ∀ (n : nat),
  sillyfun n = false.
Proof.
  intros n. unfold sillyfun.
  destruct (n =? 3) eqn:E1.
  - (* n =? 3 = true *) reflexivity.
  - (* n =? 3 = false *) destruct (n =? 5) eqn:E2.
    + (* n =? 5 = true *) reflexivity.
    + (* n =? 5 = false *) reflexivity.
  Qed.
```

After unfolding `sillyfun` in the above proof, we find that we are stuck on `if (n =? 3) then ... else ...`. But either `n` is equal to `3` or it isn't, so we can use `destruct (eqb n 3)` to let us reason about the two cases.

In general, the `destruct` tactic can be used to perform case analysis of the results of arbitrary computations. If `e` is an expression whose type is some inductively defined type `T`, then, for each constructor `c` of `T`, `destruct e` generates a subgoal in which all occurrences of `e` (in the goal and in the context) are replaced by `c`.

Exercise: 3 stars, standard (combine split)

Here is an implementation of the `split` function mentioned in chapter Poly:

```
Fixpoint split {X Y : Type} (l : list (X×Y))
  : (list X) × (list Y) :=
```

```

match l with
| [] => ([], [])
| (x, y) :: t =>
  match split t with
  | (lx, ly) => (x :: lx, y :: ly)
  end
end.

```

Prove that `split` and `combine` are inverses in the following sense:

```

Theorem combine_split : ∀ X Y (l : list (X × Y)) l₁ l₂,
  split l = (l₁, l₂) →
  combine l₁ l₂ = l.

```

Proof.

```
(* FILL IN HERE *) Admitted.
```

□

The `eqn:` part of the `destruct` tactic is optional; although we've chosen to include it most of the time, for the sake of documentation, it can often be omitted without harm.

One example where it *cannot* be omitted is when we are *destructing* compound expressions; here, the information recorded by the `eqn:` can actually be critical, and, if we leave it out, then `destruct` can erase information we need to complete a proof. For example, suppose we define a function `sillyfun1` like this:

```

Definition sillyfun1 (n : nat) : bool :=
  if n =? 3 then true
  else if n =? 5 then true
  else false.

```

Now suppose that we want to convince *Rocq* that `sillyfun1 n` yields `true` only when `n` is odd. If we start the proof like this (with no `eqn:` on the `destruct`)...

```

Theorem sillyfun1_odd_FAILED : ∀ (n : nat),
  sillyfun1 n = true →
  odd n = true.

```

Proof.

```

intros n eq. unfold sillyfun1 in eq.
destruct (n =? 3).
(* stuck... *)

```

Abort.

... then we are stuck at this point because the context does not contain enough information to prove the goal! The problem is that the substitution performed by `destruct` is quite brutal -- in this case, it throws away every occurrence of `n =? 3`, but we need to keep some memory of this expression and how it was destructed, because we need to be able to reason that, since we are assuming `n =? 3 = true` in this branch of the case analysis, it must be that `n = 3`, from which it follows that `n` is odd.

What we want here is to substitute away all existing occurrences of `n =? 3`, but at the same time add an equation to the context that records which case we are in. This is precisely what the `eqn:` qualifier does.

```

Theorem sillyfun1_odd :  $\forall$  (n : nat),
  sillyfun1 n = true  $\rightarrow$ 
  odd n = true.
Proof.
  intros n eq. unfold sillyfun1 in eq.
  destruct (n =? 3) eqn:Heqe3.

```

Now we have the same state as at the point where we got stuck above, except that the context contains an extra equality assumption, which is exactly what we need to make progress.

```

- (* e3 = true *) apply eqb_true in Heqe3.
  rewrite  $\rightarrow$  Heqe3. reflexivity.
- (* e3 = false *)

```

When we come to the second equality test in the body of the function we are reasoning about, we can use `eqn`: again in the same way, allowing us to finish the proof.

```

destruct (n =? 5) eqn:Heqe5.
+ (* e5 = true *)
  apply eqb_true in Heqe5.
  rewrite  $\rightarrow$  Heqe5. reflexivity.
+ (* e5 = false *) discriminate eq. Qed.

```

Exercise: 2 stars, standard (destruct eqn practice)

```

Theorem bool_fn_applied_thrice :
   $\forall$  (f : bool  $\rightarrow$  bool) (b : bool),
  f (f (f b)) = f b.
Proof.
  (* FILL IN HERE *) Admitted.

```

□

Review

We've now talked about many of Rocq's most fundamental tactics. We'll introduce a few more in the coming chapters, and later on we'll see some more powerful automation tactics that make Rocq help us with low-level details. But basically we've got what we need to get work done.

Here are the ones we've seen:

- `intros`: move hypotheses/variables from goal to context
- `reflexivity`: finish the proof (when the goal looks like $e = e$)
- `apply`: prove goal using a hypothesis, lemma, or constructor
- `apply ... in H`: apply a hypothesis, lemma, or constructor to a hypothesis in the context (forward reasoning)
- `apply ... with ...`: explicitly specify values for variables that cannot be determined by pattern matching
- `specialize (H ...)`: refine a hypothesis by fixing some of its variables

- *simpl*: simplify computations in the goal
- *simpl in H*: ... or a hypothesis
- *rewrite*: use an equality hypothesis (or lemma) to rewrite the goal
- *rewrite ... in H*: ... or a hypothesis
- *symmetry*: changes a goal of the form $t=u$ into $u=t$
- *symmetry in H*: changes a hypothesis of the form $t=u$ into $u=t$
- *transitivity y*: prove a goal $x=z$ by proving two new subgoals, $x=y$ and $y=z$
- *unfold*: replace a defined constant by its right-hand side in the goal
- *unfold ... in H*: ... or a hypothesis
- *destruct ... as ...*: case analysis on values of inductively defined types
- *destruct ... eqn: ...*: specify the name of an equation to be added to the context, recording the result of the case analysis
- *induction ... as ...*: induction on values of inductively defined types
- *injection ... as ...*: reason by injectivity on equalities between values of inductively defined types
- *discriminate*: reason by disjointness of constructors on equalities between values of inductively defined types
- *replace x with y*: replaces x with y everywhere for the current goal, creating a subgoal that $x = y$.
- *assert (H: e) (or assert (e) as H)*: introduce a "local lemma" e and call it H
- *generalize dependent x*: move the variable x (and anything else that depends on it) from the context back to an explicit hypothesis in the goal formula
- *f_equal*: change a goal of the form $f\ x = f\ y$ into $x = y$

Additional Exercises

Exercise: 3 stars, standard (eqb_sym)

```
Theorem eqb_sym : ∀ (n m : nat),
  (n =? m) = (m =? n).
Proof.
  (* FILL IN HERE *) Admitted.
```

□

Exercise: 3 stars, advanced, optional (eqb_sym informal)

Give an informal proof of this lemma that corresponds to your formal proof above:

Theorem: For any nats $n\ m$, $(n =? m) = (m =? n)$.

Proof:

```
(* FILL IN HERE *)
```

□

Exercise: 3 stars, standard, optional (eqb_trans)

```
Theorem eqb_trans : ∀ n m p,
  n =? m = true →
  m =? p = true →
  n =? p = true.
```

Proof.

```
(* FILL IN HERE *) Admitted.
```

□

Exercise: 3 stars, advanced (split_combine)

We proved, in an exercise above, that `combine` is the inverse of `split`. Complete the definition of `split_combine_statement` below with a property that states that `split` is the inverse of `combine`. Then, prove that the property holds.

Hint: Take a look at the definition of `combine` in *Poly*. Your property will need to account for the behavior of `combine` in its base cases, which possibly drop some list elements.

```
Definition split_combine_statement : Prop
  (* (" : Prop" means that we are giving a name to a
     logical proposition here.) *)
  (* REPLACE THIS LINE WITH ":= _your_definition_ ." *).
Admitted.
```

```
Theorem split_combine : split_combine_statement.
```

Proof.

```
(* FILL IN HERE *) Admitted.
```

```
(* Do not modify the following line: *)
```

```
Definition manual_grade_for_split_combine : option (nat×string)
:= None.
```

□

Exercise: 3 stars, advanced (filter exercise)

```
Theorem filter_exercise : ∀ (X : Type) (test : X → bool)
  (x : X) (l lf : list X),
  filter test l = x :: lf →
  test x = true.
```

Proof.

```
(* FILL IN HERE *) Admitted.
```

□

Exercise: 4 stars, advanced, especially useful (forall_exists_challenge)

Define two recursive Fixpoints, `forallb` and `existsb`. The first checks whether every element in a list satisfies a given predicate:

```
forallb odd [1;3;5;7;9] = true
forallb negb [false;false] = true
forallb even [0;2;4;5] = false
forallb (eqb 5) [] = true
```

The second checks whether there exists an element in the list that satisfies a given predicate:

```
existsb (eqb 5) [0;2;3;6] = false
existsb (andb true) [true;true;false] = true
existsb odd [1;0;0;0;0;3] = true
existsb even [] = false
```

Next, define a nonrecursive version of `existsb` -- call it `existsb'` -- using `forallb` and `negb`.

Finally, prove a theorem `existsb_existsb'` stating that `existsb'` and `existsb` have the same behavior.

```
Fixpoint forallb {X : Type} (test : X → bool) (l : list X) :
bool
  (* REPLACE THIS LINE WITH ":= _your_definition_ ." *).
Admitted.
```

```
Example test_forallb_1 : forallb odd [1;3;5;7;9] = true.
Proof. (* FILL IN HERE *) Admitted.
```

```
Example test_forallb_2 : forallb negb [false;false] = true.
Proof. (* FILL IN HERE *) Admitted.
```

```
Example test_forallb_3 : forallb even [0;2;4;5] = false.
Proof. (* FILL IN HERE *) Admitted.
```

```
Example test_forallb_4 : forallb (eqb 5) [] = true.
Proof. (* FILL IN HERE *) Admitted.
```

```
Fixpoint existsb {X : Type} (test : X → bool) (l : list X) :
bool
  (* REPLACE THIS LINE WITH ":= _your_definition_ ." *).
Admitted.
```

```
Example test_existsb_1 : existsb (eqb 5) [0;2;3;6] = false.
Proof. (* FILL IN HERE *) Admitted.
```

```
Example test_existsb_2 : existsb (andb true) [true;true;false]
= true.
Proof. (* FILL IN HERE *) Admitted.
```

```
Example test_existsb_3 : existsb odd [1;0;0;0;0;3] = true.
Proof. (* FILL IN HERE *) Admitted.
```

```
Example test_existsb_4 : existsb even [] = false.
Proof. (* FILL IN HERE *) Admitted.
```

```
Definition existsb' {X : Type} (test : X → bool) (l : list X) :
bool
  (* REPLACE THIS LINE WITH ":= _your_definition_ ." *).
Admitted.
```

```
Theorem existsb_existsb' : ∀ (X : Type) (test : X → bool) (l :  
list X),  
  existsb test l = existsb' test l.  
Proof. (* FILL IN HERE *) Admitted.  
□  
  
(* 2026-01-07 13:17 *)
```